



Universidad Tecnológica Nacional
Modalidad distancia

Trabajo Integrador

**Food Store – Sistema de Gestión de Pedidos
de Comida (Consola)**

Materia
Programación II

Alumno
Alexander Sequera
Alejandro Ruiz Diaz

Fecha
22-06-2026

Índice

Índice.....	2
Marco Teórico.....	4
1. Java - Lenguaje de Programación.....	4
2. Java - Lenguaje de Programación.....	5
2.1 Encapsulamiento.....	6
2.2 Herencia.....	6
2.3 Polimorfismo.....	7
2.4 Abstracción.....	7
3. Colecciones.....	7
4. Enums.....	9
5. Manejo de excepciones.....	11
6. Entrada/salida por consola.....	13
7. Entrada/salida por consola.....	14
Arquitectura en capas.....	14
Inyección de dependencias por constructor.....	15
Borrado lógico (soft delete).....	15
Objeto de Transferencia de Datos (DTO).....	15
Centralización de mensajes.....	15
Decisiones Técnicas y Arquitectura.....	16
Visión general del sistema.....	16
Organización en paquetes.....	17
Dependencias entre paquetes.....	18
Convenciones de nomenclatura.....	19
Modelo de dominio.....	20
Entidades del sistema.....	20
Relaciones entre entidades.....	21
Diagrama de clases.....	21
Capa de servicios y reglas de negocio.....	22
Estructura común de los servicios.....	22
Reglas de negocio aplicadas.....	22

Colaboración entre servicios.....	23
Manejo de errores.....	23
Capa de interfaz y flujo de interacción.....	23
1. Organización de los menús.....	23
2. Patrón estructural común.....	24
3. Captura de excepciones.....	24
4. Reutilización mediante utilitarios.....	24
5. Flujo típico de interacción.....	24
Menú principal.....	25
Listado de productos.....	25
Creación de un usuario.....	26
Creación de un pedido.....	26
Manejo de error: stock insuficiente.....	27
Manejo de error: dato duplicado.....	27
Dificultades y Soluciones.....	28
1. Traducción del diseño UML al código fuente.....	28
2. Detección y corrección de errores.....	28
3. Adopción de una arquitectura profesional.....	29
4. Modelado de la lógica de negocio.....	29
5. Repaso de conceptos de Java y POO.....	29
6. Coordinación del trabajo en equipo.....	29
7. Interpretación de la consigna.....	30
Anexos.....	30
Bibliografía / Webgrafía.....	30
Links.....	31

Marco Teórico

1. Java - Lenguaje de Programación

Java es un lenguaje de programación de propósito general, orientado a objetos y de alto nivel, ampliamente utilizado en la industria del software para el desarrollo de aplicaciones móviles, sistemas empresariales, videojuegos, computación en la nube y dispositivos del Internet de las Cosas (IoT). Fue creado por **James Gosling** mientras trabajaba en *Sun Microsystems*, y su primera versión pública, Java 1.0, se publicó en **1996**. El proyecto se denominaba originalmente *Oak* y fue concebido para la televisión interactiva, pero ante las limitaciones tecnológicas de la época, su enfoque se redirigió hacia un lenguaje portable que pudiera ejecutarse en múltiples dispositivos conectados a la red. A más de dos décadas de su lanzamiento, Java continúa siendo uno de los lenguajes más utilizados a nivel mundial y constituye la base del sistema operativo Android, entre otros.

Una de las características distintivas de Java es su naturaleza **multiplataforma**, principio resumido en el lema *Write Once, Run Anywhere* (WORA): el código escrito para un sistema operativo puede ejecutarse en cualquier otro sin necesidad de modificaciones, siempre que el sistema destino cuente con una implementación compatible de la Máquina Virtual de Java (JVM). Asimismo, Java se considera un lenguaje seguro, ya que permite ejecutar código en un entorno controlado que impide el acceso directo al hardware o al sistema de archivos del equipo anfitrión, y dispone de niveles de seguridad altamente configurables. A esto se suma su sintaxis derivada del lenguaje C++, lo que facilita su adopción por parte de programadores con experiencia previa en lenguajes similares, y un extenso ecosistema de bibliotecas, funciones integradas y herramientas de desarrollo que aceleran la escritura de software.

El funcionamiento de Java se distingue por combinar las estrategias tradicionales de **compilación e interpretación**. El código fuente, escrito en archivos **.java**, es compilado por el *Java Development Kit* (JDK) a un formato intermedio denominado **bytecode** (archivos **.class**), independiente de la arquitectura de hardware subyacente. Posteriormente, este bytecode es procesado por la JVM, que actúa como una capa de abstracción entre el programa y el sistema operativo: si la aplicación se ejecuta sobre Windows, la JVM interpreta el bytecode para Windows; si lo hace sobre Linux o macOS, se interpreta para esa plataforma. Esta arquitectura es la que permite la portabilidad del lenguaje y explica que un mismo archivo compilado pueda distribuirse sin modificaciones entre distintos entornos de ejecución.

En el desarrollo del presente sistema, Java fue elegido por su robustez, su madurez como lenguaje y su fuerte orientación a objetos, características que se alinean directamente con los requerimientos del proyecto. Al tratarse de una aplicación de gestión que modela entidades del mundo real —categorías, productos, usuarios y pedidos—, resultaba conveniente trabajar con un lenguaje que permitiera representar dichas entidades como **clases** con atributos y comportamientos, y establecer relaciones entre ellas mediante mecanismos como la herencia y la composición. Además, la disponibilidad de una biblioteca estándar amplia permitió aprovechar componentes ya provistos por el lenguaje, como el *Java Collections Framework* para la gestión de listas dinámicas, la clase `Scanner` para la lectura por consola y la API `java.time` para el manejo de fechas. La portabilidad propia de Java garantiza, asimismo, que el programa pueda compilarse y ejecutarse indistintamente sobre Windows, Linux o macOS sin requerir ajustes en el código fuente, lo que constituye una ventaja para su distribución y futuras extensiones.

2. Java - Lenguaje de Programación

La **Programación Orientada a Objetos (POO)** es un paradigma de programación basado en el concepto de **objetos**, entidades de software que combinan datos, llamados **atributos**, y comportamiento, representado por **métodos**, dentro de una única unidad lógica. Su propósito es representar fielmente entidades del mundo real dentro de un programa, organizando el código en torno a estos objetos en lugar de hacerlo en torno a funciones o procedimientos. Este paradigma favorece la **reutilización del código**, la **mantenibilidad** y la **escalabilidad** del software, y constituye la base de lenguajes como **Java, C++, C#, Python y Ruby**.

Los dos elementos centrales de la POO son la **clase** y el **objeto**. Una **clase** es una plantilla que define los atributos y métodos comunes a un conjunto de entidades; un **objeto** es una instancia concreta de una clase, con un estado, un comportamiento y una identidad propios. La inicialización de un objeto se realiza mediante un **constructor**, un método especial con el mismo nombre que la clase y sin tipo de retorno, que se invoca automáticamente al instanciarla. En el sistema desarrollado, las entidades del dominio, **Categoría, Producto, Usuario, Pedido y DetallePedido**, se modelaron como clases independientes, y se utilizaron **constructores parametrizados** para garantizar que ningún objeto pudiera existir sin sus datos esenciales correctamente inicializados. La clase **Pedido**, por ejemplo, recibe el usuario asociado y fija automáticamente la fecha actual, el estado inicial **PENDIENTE** y una forma de pago por defecto.

Sobre estos conceptos se erigen los **cuatro pilares** del paradigma: **encapsulamiento, herencia, polimorfismo y abstracción**.

2.1 Encapsulamiento

El **encapsulamiento** consiste en agrupar los datos y los métodos que operan sobre ellos dentro de una única unidad lógica, restringiendo el acceso directo a los atributos desde el exterior. En **Java** se implementa mediante **modificadores de acceso**, principalmente **private** y **public**, y métodos **getters** y **setters** que regulan la lectura y escritura del estado interno del objeto.

En el sistema desarrollado, todas las clases declaran sus atributos como **privados** y los exponen mediante **métodos públicos**. Las **validaciones de negocio** no se ubican dentro de los setters, sino en la **capa de servicios**, manteniendo a las entidades enfocadas exclusivamente en la representación del dato. Un ejemplo particular es el método **getDetalles()** de la clase **Pedido**, que devuelve una copia de la lista interna en lugar de la lista original, impidiendo que el código externo modifique el estado del pedido directamente.

2.2 Herencia

La **herencia** es el mecanismo que permite a una clase adquirir los atributos y métodos de otra, estableciendo una relación de tipo "**es un**" mediante la palabra reservada **extends**. La clase de la que se hereda se denomina **superclase** y la que hereda, **subclase**. **Java** admite **herencia simple, multinivel y jerárquica** entre clases, y **herencia múltiple** únicamente a través de **interfaces**.

En el sistema, se aplicó **herencia jerárquica** mediante una clase base denominada **Base**, de la cual heredan todas las entidades del dominio. Esta clase centraliza los atributos comunes, **identificador, marca de eliminación lógica y fecha de creación**, evitando duplicación de código y asegurando que cualquier modificación en la lógica compartida se realice en un único lugar.

2.3 Polimorfismo

El **polimorfismo** permite que una misma entidad, ya sea un **método u objeto**, exhiba comportamientos diferentes según el contexto. **Java** lo implementa de dos formas: la **sobrecarga de métodos (overloading)**, resuelta en tiempo de compilación cuando varios métodos comparten nombre pero difieren en sus parámetros; y la **sobreescritura de**

métodos (overriding), resuelta en tiempo de ejecución cuando una subclase redefine un método heredado.

Ambas formas se aplican en el sistema. La **sobrecarga** se observa en la clase **Base**, que ofrece dos **constructores** con distintas firmas. La **sobreescritura** se manifiesta en el método **toString()**, redefinido en cada entidad para devolver una representación textual adaptada a sus propios atributos, y en los métodos **equals()** y **hashCode()** de la clase **Producto**, sobreescritos para permitir comparaciones por identificador en lugar de por referencia de memoria.

2.4 Abstracción

La **abstracción** consiste en exponer únicamente las características esenciales de un objeto, ocultando los detalles de su implementación interna. En **Java** se logra mediante **clases abstractas**, que no pueden instanciarse directamente y pueden contener métodos sin implementación, e **interfaces**, que definen un contrato de métodos que las clases implementadoras se comprometen a cumplir.

El sistema aprovecha ambas construcciones. La clase **Base** fue declarada como **abstract**, ya que no tiene sentido crear un objeto Base genérico: su existencia se justifica únicamente como modelo común para las entidades concretas. Por otra parte, la interfaz **Calculable** declara el método **calcularTotal()** y es implementada por la clase **Pedido**, que provee la lógica concreta del cálculo del total. Este diseño es **extensible**: cualquier entidad futura que requiera un cálculo similar, como una factura o un presupuesto, podría implementar la misma interfaz sin modificar el código existente.

3. Colecciones

Una **colección** representa un grupo de objetos, conocidos como **elementos**, agrupados dentro de una estructura que permite su almacenamiento y manipulación conjunta. **Java** proporciona el **Java Collections Framework**, un conjunto de interfaces y clases incluido en su biblioteca estándar que ofrece **estructuras de datos** genéricas y reutilizables, junto con algoritmos optimizados para operar sobre ellas. La interfaz raíz del framework es **Collection**, y de ella derivan tres subinterfaces principales: **Set**, que no admite elementos duplicados; **Map**, que asocia claves únicas a valores; y **List**, que representa una sucesión ordenada de elementos accesibles mediante un índice y que sí permite duplicados. Cada una de estas interfaces cuenta con múltiples implementaciones que se diferencian en su rendimiento y en sus garantías de orden.

Dentro de la interfaz **List**, la implementación más utilizada es **ArrayList**, basada en un arreglo redimensionable que ofrece el mejor rendimiento en la mayoría de los casos de uso. La interfaz expone, además, las operaciones heredadas de **Collection**, como **add()**, **remove()**, **contains()**, **size()**, **isEmpty()** e **iterator()**, entre otras, lo que permite manipular cualquier colección de manera uniforme con independencia de su implementación subyacente.

En el sistema desarrollado, el almacenamiento en memoria se resolvió íntegramente mediante la combinación **List + ArrayList**. Cada uno de los cuatro servicios mantiene su propia colección interna, declarada como **final** y accesible únicamente desde la clase contenedora: **List** en **CategoriaServices**, **List** en **ProductoServices**, **List** en **UsuarioServices** y **List** en **PedidosServices**. Estas colecciones actúan como un **repositorio en memoria** de las entidades del dominio durante la ejecución del programa, y son las que sostienen la totalidad de las operaciones del sistema: **alta**, **baja**, **modificación**, **búsqueda** y **listado**.

La elección de **List** sobre las otras dos interfaces principales se fundamenta en tres motivos. En primer lugar, las entidades del sistema, **productos**, **usuarios** y **pedidos**, pueden compartir datos similares pero conservan identidades únicas a través de su atributo **id**, por lo que la restricción de unicidad propia de **Set** no aporta valor. En segundo lugar, las operaciones predominantes son la **iteración secuencial** para realizar búsquedas, filtrados y listados, por ejemplo, recorrer los productos para obtener únicamente los que poseen stock disponible o pertenecen a una categoría específica, lo cual se ajusta al rendimiento óptimo de **ArrayList**. Y en tercer lugar, el **orden de inserción** resulta suficiente para las necesidades del sistema, sin requerir ordenamiento automático ni acceso por clave.

El framework también se aprovechó a través de su **polimorfismo**: en todas las declaraciones se utiliza la interfaz **List** en lugar de la implementación concreta **ArrayList**. Esta práctica permitiría, en el futuro, sustituir la implementación subyacente, por ejemplo por **LinkedList** o por una colección sincronizada, sin necesidad de modificar el código de los servicios que la consumen. Asimismo, los métodos de listado de cada servicio, **listarTodos()**, **listarActivos()**, **listarConStock()** y **listarPorCategoria()**, retornan una nueva instancia de **ArrayList** construida a partir de la colección interna, mediante la expresión **new ArrayList<>(coleccion)**. Esta **copia defensiva** protege el estado interno del servicio frente a modificaciones externas no controladas: el código que recibe la lista puede recorrerla, filtrarla o modificarla sin afectar el repositorio original, reforzando así el principio de **encapsulamiento** expuesto en la sección anterior.

Finalmente, las operaciones del framework se utilizan de manera intensiva en toda la **capa de servicios**. El método **add()** permite incorporar nuevas entidades al repositorio cuando se ejecuta una operación de **alta**; los bucles **for-each** recorren las colecciones para resolver búsquedas por identificador, validar duplicados por nombre o correo electrónico y filtrar elementos según su estado de **eliminación lógica**. La clase **Pedido** también utiliza un **ArrayList** para representar los ítems que componen cada pedido, lo que demuestra que el uso de colecciones no se limita a la capa de servicios, sino que forma parte integral del **modelo de dominio**.

4. Enums

Un **tipo enumerado** es un tipo de dato especial que permite definir un conjunto fijo y limitado de **constantes** con nombre, conocidas en tiempo de compilación. Se utilizan cuando una variable debe tomar su valor dentro de un grupo cerrado de opciones predefinidas, como los días de la semana, los puntos cardinales o los estados de un proceso. En **Java**, los tipos enumerados se declaran mediante la palabra reservada **enum** y, por convención, los nombres de sus constantes se escriben en mayúsculas.

A diferencia de las constantes tradicionales declaradas con **static final**, los **enum** proveen **seguridad de tipo** en tiempo de compilación: el compilador garantiza que una variable de tipo enumerado únicamente pueda recibir uno de los valores definidos, lo que evita la asignación accidental de valores inválidos. Internamente, cada **enum** define una clase que extiende implícitamente **java.lang.Enum** y puede contar con atributos, constructores y métodos propios cuando se requiere asociar información o comportamiento a cada constante. Asimismo, los enumerados se integran de manera natural con la sentencia **switch**, lo que permite escribir bloques de decisión más limpios y legibles.

En el sistema desarrollado se definieron tres **tipos enumerados**, ubicados en el paquete **enums**, cada uno asociado a un atributo concreto de las entidades del dominio:

- **Rol**: con las constantes **ADMIN** y **USUARIO**, representa el tipo de usuario dentro del sistema y se utiliza como atributo de la clase **Usuario**.
- **Estado**: con las constantes **PENDIENTE**, **CONFIRMADO**, **TERMINADO** y **CANCELADO**, modela el ciclo de vida de un pedido. Al crearse un pedido se asigna automáticamente el estado **PENDIENTE**, y desde el menú correspondiente puede transicionar al resto de los estados.

- **FormaPago**: con las constantes **EFFECTIVO**, **TARJETA** y **TRANSFERENCIA**, representa el medio de pago seleccionado al momento de generar un pedido.

La decisión de utilizar **enumerados** en estos tres casos respondió a un criterio común: todos los atributos involucrados aceptan únicamente un conjunto cerrado y conocido de valores. Modelarlos como tipos enumerados, en lugar de hacerlo mediante **cadenas de texto** o **constantes numéricas**, aporta varias ventajas concretas al sistema. En primer lugar, elimina la posibilidad de **valores inválidos**: el compilador impide que un pedido reciba un estado inexistente o que un usuario tenga un rol no contemplado, error que con cadenas de texto solo se detectaría en tiempo de ejecución. En segundo lugar, mejora la **legibilidad** del código, ya que expresiones como **Estado.PENDIENTE** o **FormaPago.TRANSFERENCIA** resultan inmediatamente comprensibles, a diferencia de identificadores numéricos arbitrarios. Y en tercer lugar, facilita el **mantenimiento**: incorporar un nuevo estado o forma de pago requiere modificar un único archivo, y el compilador señalará automáticamente los lugares del código donde sea necesario contemplar el nuevo valor.

La integración de los enumerados con la lógica del sistema se aprecia particularmente en los **menús** de la capa de interfaz: al solicitar al usuario que seleccione un estado o una forma de pago, se construyen **menús dinámicos** a partir de las constantes disponibles, y la entrada se traduce directamente al valor enumerado correspondiente, que luego se pasa como argumento al método del servicio. De esta manera, las firmas de los métodos, por ejemplo **editar(Long idPedido, Estado estado, FormaPago formaPago)**, resultan **autodescriptivas** y libres de ambigüedad. Los enumerados utilizados en el proyecto se mantuvieron en su forma más simple, sin atributos ni métodos asociados, dado que su única función es representar un conjunto de valores discretos sin información adicional vinculada a cada uno de ellos.

5. Manejo de excepciones

En el contexto de la programación, una **excepción** es un evento que se produce durante la ejecución de un programa y que interrumpe su flujo normal. Las excepciones pueden originarse por múltiples causas: entradas inválidas por parte del usuario, recursos no disponibles, errores de programación o un uso incorrecto de una interfaz de programación. A diferencia de la programación procedimental tradicional, donde los errores suelen señalizarse mediante valores de retorno, **Java** proporciona un mecanismo dedicado al **manejo de excepciones** que permite separar la lógica principal del programa de la lógica de tratamiento de errores, favoreciendo la robustez del código.

En **Java**, todas las excepciones son objetos que descienden de la clase **Throwable** y se organizan en una jerarquía. De ella derivan dos ramas principales: la clase **Error**, que representa fallos graves de los que el programa generalmente no puede recuperarse, como un desbordamiento de memoria, y la clase **Exception**, que agrupa las situaciones excepcionales recuperables. Dentro de esta segunda rama existe una distinción fundamental entre **excepciones verificadas (checked exceptions)**, que el compilador obliga a manejar explícitamente, y **excepciones no verificadas (unchecked exceptions)**, que descienden de **RuntimeException** y suelen estar asociadas a errores de programación detectables únicamente en tiempo de ejecución.

El manejo de excepciones se articula sobre cinco **palabras reservadas**. El bloque **try** delimita el fragmento de código en el que puede producirse una excepción; el bloque **catch** define el código que se ejecutará para tratarla, capturando el tipo de excepción correspondiente; el bloque opcional **finally** contiene instrucciones que se ejecutan siempre, con independencia de si se produjo o no una excepción, y suele utilizarse para liberar recursos. A estas se suman **throw**, que permite generar manualmente una excepción, y **throws**, que se incluye en la firma de un método para declarar que dicho método puede propagar una excepción hacia quien lo invoque.

En el sistema desarrollado, el manejo de excepciones se aplicó siguiendo dos lineamientos principales: definir un conjunto de **excepciones personalizadas** propias del dominio y capturarlas en la **capa de interfaz**, manteniendo a la **capa de servicios** libre de bloques **try/catch**. Para ello se creó el paquete **exception**, que contiene cuatro clases que extienden **RuntimeException**, cada una asociada a un tipo de error de negocio claramente identificable:

- **DatoInvalidoException**, lanzada cuando un dato es nulo, vacío o no cumple con las reglas de validación, por ejemplo, un precio negativo, una cantidad menor o igual a cero o un nombre vacío.
- **DatoDuplicadoException**, lanzada al intentar registrar una entidad cuyo identificador único ya existe, como un producto con un nombre ya utilizado o un usuario con un correo electrónico ya registrado.
- **DatoInexistenteException**, lanzada cuando una búsqueda por identificador no encuentra ningún registro activo correspondiente.
- **StockInsuficienteException**, lanzada al intentar crear un pedido cuya cantidad solicitada supera el stock disponible del producto.

La elección de extender **RuntimeException**, y no **Exception**, respondió a una decisión de diseño deliberada: estas excepciones representan **violaciones de reglas de negocio**, no fallos técnicos del sistema. Obligar a cada método de servicio a declararlas mediante **throws** habría contaminado las firmas sin aportar valor real, dado que el manejo de estos errores se concentra en un único punto: la **capa de interfaz**. De esta manera, los servicios pueden lanzar excepciones libremente mediante la instrucción **throw new** cuando detectan una condición inválida, sin necesidad de propagarlas explícitamente.

La captura de las excepciones se realiza íntegramente en los menús de la capa **ui**, donde cada opción que invoca un servicio envuelve la llamada en un bloque **try/catch**. El patrón aplicado consiste en capturar las excepciones del dominio para mostrar al usuario un mensaje claro sin interrumpir la ejecución del programa y mantener separadas de la lógica principal todas las acciones de recuperación. Un ejemplo característico se observa al crear un **Pedido**: el servicio valida secuencialmente la existencia del usuario, la validez de cada ítem y la disponibilidad de stock; ante cualquier incumplimiento lanza la excepción correspondiente, que el menú captura y traduce en un mensaje informativo, permitiendo al usuario corregir su entrada y reintentar la operación.

Adicionalmente, los textos de los **mensajes de error** no se escriben en línea dentro del código, sino que se centralizan en clases dedicadas del paquete **utils**, como **MensajesGenerales**, **MensajesProducto** y **MensajesPedidos**. Esta organización facilita el **mantenimiento** del sistema y la eventual traducción de los mensajes, además de evitar duplicación de cadenas a lo largo del código.

6. Entrada/salida por consola

La **Entrada/Salida (Input/Output o I/O)** es el conjunto de mecanismos que permiten a un programa comunicarse con el exterior: leer datos provenientes de una fuente, como el teclado, un archivo o la red, y escribir resultados hacia un destino, como la pantalla, un archivo o un dispositivo periférico. **Java** agrupa estas funcionalidades en el paquete **java.io**, que provee un conjunto amplio de clases y **flujos (streams)** preparados para manejar tanto datos binarios como datos de texto.

Para la comunicación con la consola, **Java** pone a disposición tres flujos predeterminados accesibles a través de la clase **System**: **System.in**, el flujo de entrada estándar utilizado para leer datos del teclado; **System.out**, el flujo de salida estándar utilizado para mostrar resultados en pantalla; y **System.err**, un flujo de salida específico destinado a mensajes de error, separado de la salida normal del programa. Sobre

System.out se invocan habitualmente tres métodos: **print()**, que imprime el texto y mantiene el cursor en la misma línea; **println()**, que imprime el texto y desplaza el cursor al inicio de la línea siguiente; y **printf()**, que permite generar salidas con formato a partir de patrones similares a los del lenguaje C.

Si bien **System.in** permite leer datos directamente, los métodos que expone operan a nivel de bytes individuales, lo cual resulta poco práctico para procesar entradas reales del usuario. Por este motivo, la práctica habitual consiste en envolverlo en una clase de mayor nivel que ofrezca una interfaz más cómoda. La clase **Scanner**, del paquete **java.util**, cumple precisamente esa función: lee la entrada del flujo subyacente, la fracciona en **tokens** y los convierte automáticamente al tipo de dato solicitado mediante métodos como **nextInt()**, **nextLine()**, **nextDouble()** o **next()**.

En el sistema desarrollado, toda la comunicación con el usuario se resuelve por consola: el programa imprime menús y mensajes mediante **System.out** y lee las respuestas del usuario mediante una instancia de **Scanner** construida a partir de **System.in**. Esta decisión es coherente con la naturaleza del proyecto, concebido como una aplicación de gestión sin interfaz gráfica, y permite ejecutarlo en cualquier entorno, **Windows**, **Linux** o **macOS**, sin dependencias adicionales más allá de la **JVM**.

La gestión del **Scanner** se diseñó siguiendo el principio de **instancia única**: en lugar de crear un **Scanner** distinto en cada menú o en cada operación, se instancia una única vez en el constructor de la clase **MenuPrincipal** y se propaga al resto de los menús, **MenuCategoria**, **MenuProducto**, **MenuUsuarios** y **MenuPedidos**, a través de sus respectivos constructores. Esta práctica responde a una limitación conocida del lenguaje: cerrar un **Scanner** asociado a **System.in** cierra también el flujo subyacente y lo deja inutilizable durante el resto de la ejecución del programa. Mantener una única instancia compartida evita ese problema y, al mismo tiempo, reduce el uso de recursos.

La lectura y la escritura no se realizan directamente desde cada menú, sino que se centralizan en clases utilitarias ubicadas en el paquete **utils**. La clase **UtilsMenu** contiene el método **leerOpcion()**, encargado de leer la opción seleccionada por el usuario dentro de un menú, validando que se trate de un valor numérico válido. La clase **UtilsGeneral** agrupa el resto de las operaciones de lectura comunes: lectura de identificadores numéricos mediante **leerId()**, pausas para confirmar la lectura de un mensaje mediante **esperarEnter()**, e impresión de títulos y separadores. La clase **UtilsListar**, por su parte, concentra la salida tabular de las distintas entidades, **usuarios**, **productos** y **pedidos**, garantizando que el formato visual sea consistente a lo largo de todo el sistema. Esta organización evita duplicar

código de **entrada/salida** en cada menú y facilita aplicar cambios globales, por ejemplo, modificar el formato de los listados, en un único lugar.

Finalmente, los menús siguen un **patrón estructural** común basado en un bucle **while** que se repite mientras el usuario no seleccione la opción de salida. Dentro de cada iteración, el programa imprime el menú correspondiente mediante **System.out**, lee la opción ingresada y la deriva a una sentencia **switch** que invoca el método asociado a cada operación. Este patrón se replica en los cinco menús del sistema, lo que aporta **uniformidad** en la experiencia del usuario y **consistencia** en la estructura del código.

7. Entrada/salida por consola

Más allá del uso del paradigma orientado a objetos, el diseño del sistema incorporó un conjunto de **patrones de diseño** y **buenas prácticas de ingeniería de software** que mejoran su **mantenibilidad** y **extensibilidad**. A continuación se describen los principales.

Arquitectura en capas

La **arquitectura en capas** organiza el código en niveles con responsabilidades delimitadas y dependencias unidireccionales. El sistema se estructuró en cuatro capas reflejadas en los paquetes **ui**, **services**, **entities** y las capas transversales **enums**, **exception** y **utils**, que contienen elementos compartidos. Esta separación permite, por ejemplo, sustituir el almacenamiento en memoria por una base de datos modificando únicamente la **capa de servicios**.

Inyección de dependencias por constructor

La **inyección de dependencias por constructor** consiste en entregar a un componente sus dependencias desde el exterior, a través de su constructor, en lugar de instanciarlas internamente. En el sistema, **ProductoServices** recibe una instancia de **CategoriaServices**, y **PedidosServices** recibe **UsuarioServices** y **ProductoServices**. La composición global se centraliza en la clase **MenuPrincipal**, que actúa como único punto de configuración. Esta práctica reduce el **acoplamiento** entre clases y facilita su prueba aislada.

Borrado lógico (soft delete)

El **borrado lógico (soft delete)** consiste en marcar los registros como eliminados mediante una bandera, en lugar de removerlos físicamente. Se implementó a través del atributo

eliminado, heredado de la clase **Base**. Esta decisión preserva la **integridad histórica**: un pedido cerrado en el pasado puede seguir mostrando sus productos asociados aun cuando dichos productos hayan sido dados de baja con posterioridad.

Objeto de Transferencia de Datos (DTO)

Un **Objeto de Transferencia de Datos (DTO)** es un objeto cuya única función es transportar datos entre capas, sin contener lógica de negocio. En el sistema se aplicó mediante la clase **ItemPedido**: cuando el usuario construye un pedido, dispone únicamente de pares formados por **id de producto** y **cantidad**, mientras que la entidad **DetallePedido** requiere un objeto **Producto** resuelto y un **subtotal** calculado. **ItemPedido** actúa como intermediario, transportando los datos crudos desde el menú hacia el servicio, que se encarga de la transformación.

Centralización de mensajes

La **centralización de mensajes** consiste en concentrar los textos reutilizables del sistema en un único lugar. En el proyecto, todos los **mensajes de error** y los mensajes repetitivos se concentran en clases dedicadas del paquete **utils**, como **MensajesGenerales**, **MensajesProducto** y **MensajesPedidos**. Esta organización evita la dispersión de cadenas a lo largo del código y facilita su **mantenimiento** y su eventual traducción.

Decisiones Técnicas y Arquitectura

Visión general del sistema

El sistema desarrollado es una aplicación de consola orientada a la **gestión integral** de un comercio de tipo **food store**. Su propósito es permitir la administración de cuatro **entidades** centrales del negocio, **categorías**, **productos**, **usuarios** y **pedidos**, a través de un **menú interactivo**, aplicando reglas de validación coherentes con un sistema real de gestión: **control de stock**, **unicidad de identificadores naturales**, **validación de formas de pago** y **seguimiento del estado** de cada pedido.

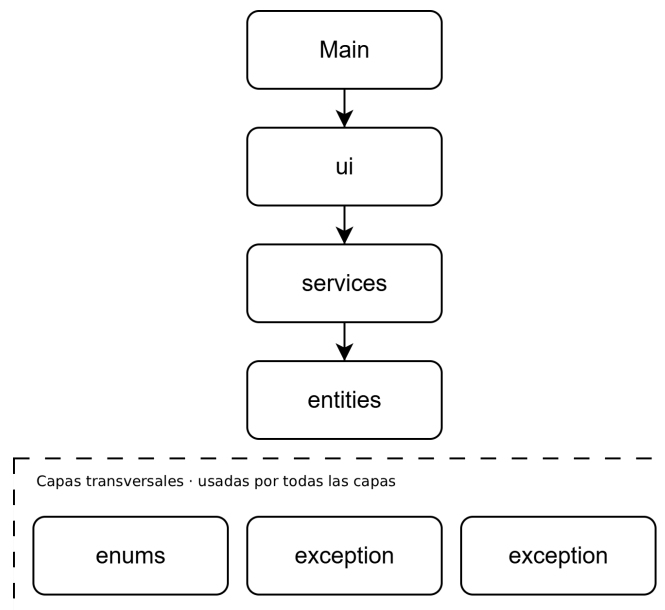
La aplicación se desarrolló íntegramente en **Java**, sin recurrir a **frameworks** ni bibliotecas externas más allá de la **API estándar** del lenguaje. Esta decisión responde al carácter académico del proyecto, en el que se priorizó la aplicación rigurosa de los conceptos del **paradigma orientado a objetos** y de la biblioteca estándar de Java, sin delegar la solución de problemas estructurales a herramientas de terceros. La **persistencia de los datos** se resuelve en memoria mediante el uso de **colecciones**, lo que implica que la información existe únicamente durante la ejecución del programa. Esta limitación es deliberada y permite concentrar el esfuerzo de diseño en el **modelado del dominio** y en la **lógica de negocio**, dejando abierta la posibilidad de incorporar una **capa de persistencia** en versiones futuras sin modificar el resto del sistema.

Funcionalmente, el sistema se organiza en cuatro **módulos**, accesibles desde un **menú principal**:

- **Módulo de Categorías:** permite crear, listar, editar y eliminar las categorías a las que se asocian los productos del catálogo.
- **Módulo de Productos:** gestiona el catálogo, contemplando atributos como **nombre**, **descripción**, **precio**, **stock**, **imagen**, **disponibilidad** y **categoría** asociada.
- **Módulo de Usuarios:** administra los usuarios del sistema, diferenciando entre los roles **ADMIN** y **USUARIO**.
- **Módulo de Pedidos:** centraliza el proceso de creación de pedidos, contemplando la selección del usuario solicitante, el agregado de uno o más productos con sus respectivas cantidades, la elección de la **forma de pago** y el **cálculo automático del total**. Incluye además la actualización del **estado del pedido** a lo largo de su ciclo de vida.

El sistema fue diseñado siguiendo una **arquitectura en capas**, con una clara separación entre la **interfaz de usuario**, la **lógica de negocio** y el **modelo de dominio**. Las secciones siguientes detallan esta organización, presentando primero la **estructura de paquetes**, luego el **modelo de entidades** y sus relaciones, y finalmente las decisiones tomadas en cada capa.

Organización en paquetes



La **estructura del proyecto** se organizó en siete **paquetes**, cada uno con una responsabilidad claramente definida dentro de la arquitectura del sistema. Esta organización refleja la **separación por capas** descrita en el **Marco Teórico** y constituye la primera línea de defensa frente a la dispersión de responsabilidades a medida que el código crece.

La siguiente tabla resume el contenido y la función de cada paquete:

Paquete	Responsabilidad	Contenido principal
entities	Modelo del dominio. Representa las entidades del negocio con sus atributos y comportamientos básicos.	Base, Categoria, Producto, Usuario, Pedido, DetallePedido , interfaz Calculable .
enums	Tipos enumerados que representan conjuntos cerrados de valores válidos.	Estado, FormaPago, Rol .

services	Lógica de negocio. Centraliza las validaciones, las reglas del dominio y la gestión de las colecciones en memoria.	CategoriaServices, ProductoServices, UsuarioServices, PedidosServices, DTO ItemPedido.
ui	Capa de presentación. Gestiona la interacción con el usuario por consola mediante menús.	MenuPrincipal, MenuCategoria, MenuProducto, MenuUsuarios, MenuPedidos.
exception	Excepciones personalizadas del dominio.	DatoInvalidoException, DatoDuplicadaException, DatoInexistenteException, StockInsuficienteException.
utils	Funciones utilitarias y mensajes centralizados, compartidos por las restantes capas.	UtilsGeneral, UtilsMenu, UtilsListar, MensajesGenerales y las clases de mensajes por entidad.
(raíz)	Punto de entrada del programa.	Main.

Dependencias entre paquetes

El sistema respeta el principio de **dependencias unidireccionales**: cada capa depende únicamente de las que se encuentran por debajo. Esto evita **ciclos de dependencias** entre paquetes y permite modificar una capa sin impactar en las superiores.

El flujo de dependencias se resume de la siguiente manera:

- **ui** depende de **services**.
- **services** depende de **entities**, **enums** y **exception**.
- **utils** es utilizado por todas las capas.

La **capa de interfaz (ui)** depende de la **capa de servicios (services)** para delegar las operaciones; la **capa de servicios** depende de las entidades (**entities**) y de los enumerados (**enums**) para construirlas y manipularlas; y todas las capas pueden lanzar

excepciones definidas en **exception** y utilizar funciones de **utils**. En ningún caso una capa inferior depende de una superior: las entidades, por ejemplo, desconocen por completo la existencia de los servicios, y los servicios desconocen los menús que los invocan.

Esta separación es la que permite afirmar que el sistema es **extensible**: incorporar una **capa de persistencia** en una base de datos requeriría modificar únicamente la **capa de servicios**, dejando intactas las entidades y los menús. Del mismo modo, sustituir la interfaz de consola por una **interfaz gráfica** afectaría exclusivamente al paquete **ui**, sin alterar la lógica de negocio.

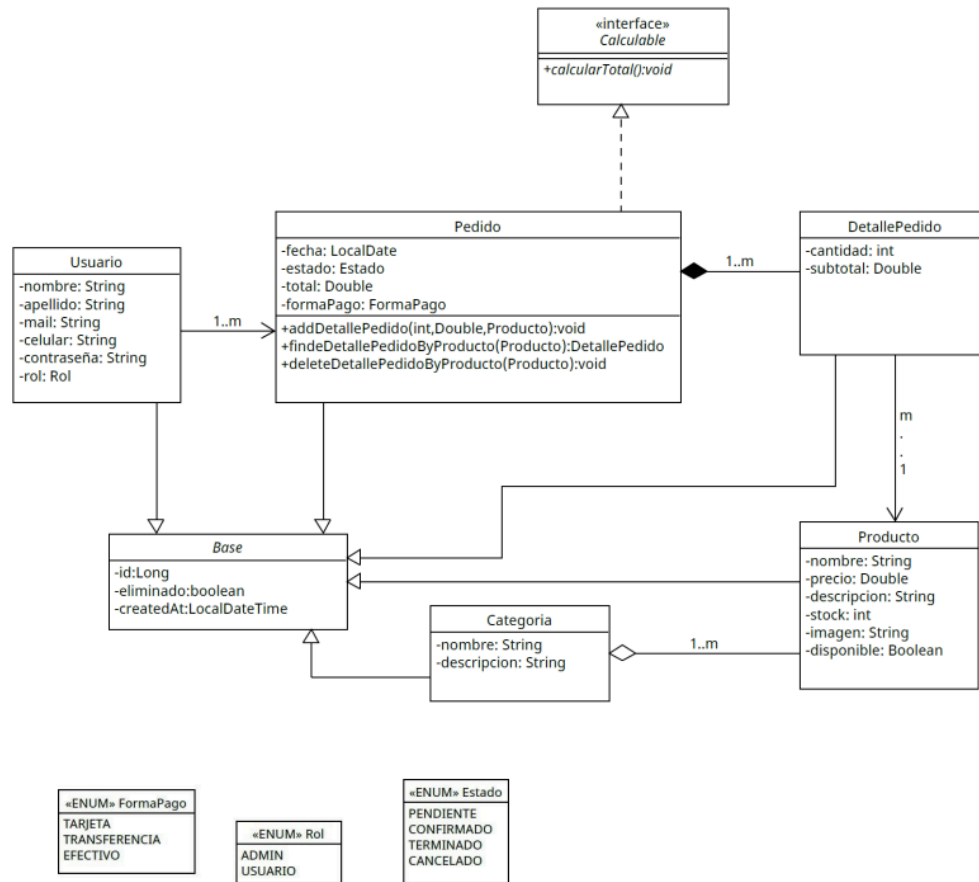
Convenciones de nomenclatura

A nivel de nomenclatura, se adoptaron las convenciones estándar del lenguaje **Java**:

- **Paquetes:** nombres en minúsculas.
- **Clases:** nombres en **PascalCase**.
- **Métodos y atributos:** nombres en **camelCase**.
- **Constantes**, incluidos los valores de los **tipos enumerados**: nombres en mayúsculas con guion bajo.

Adicionalmente, las clases de servicio se denominaron con el sufijo **Services**, por ejemplo **ProductoServices**, y las clases de menú con el prefijo **Menu**, por ejemplo **MenuProducto**, lo que permite identificar visualmente a qué capa pertenece cada clase con solo leer su nombre.

Modelo de dominio



El **modelo de dominio** del sistema está compuesto por las clases ubicadas en el paquete **entities**, que representan los conceptos centrales del negocio: **categorías**, **productos**, **usuarios**, **pedidos** y los ítems que los componen. Todas las entidades heredan de la clase abstracta **Base**, que centraliza los atributos comunes, **identificador**, **marca de eliminación lógica** y **fecha de creación**, mientras que la interfaz **Calculable** define el contrato para entidades que requieren un cálculo de total, implementada únicamente por **Pedido**.

Entidades del sistema

Entidad	Función	Atributos principales
Categoría	Agrupar productos del catálogo.	nombre, descripción

Producto	Representa un ítem vendible.	nombre, precio, stock, imagen, disponible, categoría
Usuario	Modela las personas que operan el sistema.	nombre, apellido, mail, celular, contrasena, rol
Pedido	Representa una orden generada por un usuario.	fecha, estado, total, formaPago, detalles, usuario
DetallePedido	Línea de pedido: asocia un producto con una cantidad.	cantidad, subtotal, producto

Relaciones entre entidades

Las relaciones que articulan el modelo son las siguientes:

- **Producto - Categoría (asociación muchos a uno):** cada producto pertenece a una única categoría, mientras que una categoría puede agrupar múltiples productos.
- **Pedido - Usuario (asociación muchos a uno):** cada pedido está vinculado al usuario que lo generó. Un mismo usuario puede tener múltiples pedidos.
- **Pedido - DetallePedido (composición uno a muchos):** cada pedido contiene uno o más detalles. Se trata de una relación de **composición**, dado que los detalles no tienen sentido fuera del pedido al que pertenecen: si el pedido desaparece, sus detalles también.
- **DetallePedido - Producto (asociación muchos a uno):** cada detalle referencia el producto pedido. Esta relación permite mantener la **consistencia histórica** del pedido: aunque el producto sea posteriormente dado de baja, el detalle conserva la referencia y el pedido original sigue mostrando su composición correcta.

Diagrama de clases

La estructura completa del modelo se representa gráficamente en la **Figura 2**, donde se observa la **jerarquía de herencia** hacia **Base**, la implementación de **Calculable** por

parte de **Pedido**, y las cuatro relaciones de **asociación** y **composición** que vinculan a las cinco entidades concretas.

Capa de servicios y reglas de negocio

La **capa de servicios** concentra la **lógica de negocio** del sistema. Cada entidad del dominio cuenta con un servicio dedicado, **CategoriaServices**, **ProductoServices**, **UsuarioServices** y **PedidosServices**, responsable de gestionar la colección en memoria correspondiente y de aplicar las reglas que garantizan la **consistencia de los datos**.

Estructura común de los servicios

Los cuatro servicios siguen un mismo **patrón estructural**, lo que aporta uniformidad y previsibilidad al código:

- Mantienen una colección **List** privada como repositorio interno.
- Gestionan un contador **idCounter** para asignar identificadores únicos.
- Exponen los métodos **buscarPorId()**, **listarTodos()**, **listarActivos()**, **crear()**, **editar()** y **eliminar()**, con variantes específicas según la entidad. Por ejemplo, **ProductoServices** añade **listarConStock()** y **listarPorCategoria()**.

Reglas de negocio aplicadas

Cada servicio centraliza las validaciones que protegen la **integridad del modelo**. Las reglas más representativas son las siguientes:

- **Unicidad de identificadores naturales:** **CategoriaServices** y **ProductoServices** impiden registrar dos elementos con el mismo nombre; **UsuarioServices** aplica la misma restricción sobre el correo electrónico.
- **Validación de valores obligatorios y rangos:** los campos nulos, vacíos o fuera de rango, como un precio negativo, un stock negativo o una cantidad menor o igual a cero, son rechazados al momento de la creación o edición.
- **Control de stock:** al crear un pedido, **PedidosServices** verifica que cada producto solicitado tenga stock suficiente. De cumplirse esta condición, descuenta automáticamente las unidades vendidas; en caso contrario, lanza una **StockInsuficienteException**.
- **Integridad referencial:** un producto solo puede asociarse a una categoría que exista y no esté marcada como eliminada.

- **Borrado lógico transversal:** todas las búsquedas y listados ignoran sistemáticamente los registros con **eliminado = true**, garantizando que las entidades dadas de baja no contaminen los resultados.

Colaboración entre servicios

Los servicios cooperan entre sí cuando una operación involucra a más de una entidad. La creación de un **Pedido** es el caso más representativo: **PedidosServices** invoca a **UsuarioServices** para resolver el usuario solicitante y a **ProductoServices** para validar cada producto y descontar su stock. Esta colaboración se resuelve mediante **inyección de dependencias por constructor**, lo que mantiene a cada servicio desacoplado de la forma concreta en que los demás se inicializan.

Manejo de errores

Las violaciones de las **reglas de negocio** se traducen en el lanzamiento de las **excepciones personalizadas** del paquete **exception**, **DatoInvalidoException**, **DatoDuplicadaException**, **DatoInexistenteException** y **StockInsuficienteException**. Los servicios no capturan estas excepciones, sino que las propagan hacia la **capa de interfaz**, donde se traducen en mensajes comprensibles para el usuario sin interrumpir la ejecución del programa.

Capa de interfaz y flujo de interacción

La **capa de interfaz**, ubicada en el paquete **ui**, es la responsable de toda la comunicación con el usuario. Su función es presentar los **menús**, capturar la entrada del teclado, delegar las operaciones en la **capa de servicios** y mostrar los resultados, ya sea información solicitada o mensajes de error. La interfaz se diseñó íntegramente para consola, sin recurrir a bibliotecas gráficas, lo que garantiza la **portabilidad** del sistema entre distintos sistemas operativos.

1. Organización de los menús

La capa está compuesta por cinco clases: **MenuPrincipal**, que actúa como punto de entrada y enruta hacia los módulos del sistema, y **MenuCategoria**, **MenuProducto**, **MenuUsuarios** y **MenuPedidos**, que gestionan cada uno la interacción con un módulo específico. Esta organización mantiene una correspondencia directa entre **menús** y **servicios**: cada menú consume el servicio de su entidad correspondiente y, cuando es

necesario, servicios auxiliares. Por ejemplo, **MenuProducto** utiliza también **CategoriaServices** para permitir la selección de la categoría durante el alta de un producto.

2. Patrón estructural común

Los cinco menús siguen un mismo **patrón estructural**, lo que aporta uniformidad al sistema y facilita la navegación del usuario:

- Un bucle **while** que se ejecuta mientras el usuario no seleccione la opción de salida.
- Una llamada al método **mostrar()**, que imprime las opciones disponibles mediante **System.out**.
- La lectura de la opción ingresada mediante **UtilsMenu.leerOpcion()**.
- Una sentencia **switch** que deriva a un método específico para cada opción, generalmente nombrado con el prefijo **opcion_**.
- La **opción 0**, presente en todos los menús, que rompe el bucle y devuelve el control al menú superior, o finaliza el programa en el caso de **MenuPrincipal**.

3. Captura de excepciones

Como se mencionó en la sección anterior, los servicios propagan las **excepciones del dominio** sin capturarlas. Es en la **capa de interfaz** donde se interceptan: cada método de menú que invoca a un servicio envuelve la llamada en un bloque **try/catch** que captura las excepciones esperadas y muestra al usuario un **mensaje informativo**, permitiéndole corregir su entrada y reintentar la operación sin que el programa finalice de forma inesperada.

4. Reutilización mediante utilitarios

Para evitar la duplicación de código entre los cinco menús, las operaciones repetitivas se centralizan en clases utilitarias del paquete **utils**: **UtilsMenu** para la lectura de opciones validadas, **UtilsGeneral** para operaciones comunes como la lectura de identificadores, las pausas y la impresión de títulos, y **UtilsListar** para la presentación tabular de las entidades. De este modo, una modificación en el formato visual de los listados o en la forma de validar una entrada se aplica de manera uniforme a todo el sistema desde un único lugar.

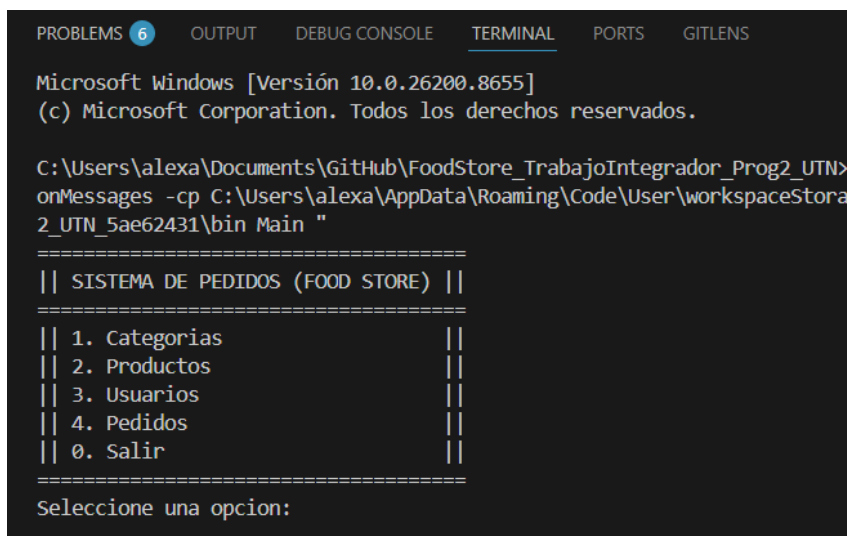
5. Flujo típico de interacción

Un flujo completo de interacción puede ilustrarse con la creación de un **Pedido**. El usuario selecciona la opción correspondiente desde el **MenuPrincipal**, lo que invoca a **MenuPedidos**. Allí elige el usuario solicitante, agrega los productos deseados especificando sus cantidades, selecciona una **FormaPago** y confirma la operación. Cada uno de estos pasos involucra una llamada a **PedidosServices**, que aplica las **reglas de negocio** descritas en la sección anterior y, en caso de éxito, devuelve el pedido creado para que el menú muestre su resumen final. Si se produce un error, como **StockInsuficienteException**, **DatoInvalidoException** o **DatoInexistenteException**, la excepción correspondiente se captura en el menú y se traduce en un mensaje claro, sin interrumpir la sesión del usuario.

6. Capturas del sistema en funcionamiento

A continuación se presentan **capturas del sistema en funcionamiento**, que ilustran las principales funcionalidades descritas en las secciones anteriores. Las imágenes fueron obtenidas durante la ejecución del programa en consola, utilizando **datos de prueba** representativos del dominio.

Menú principal



```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
Microsoft Windows [Versión 10.0.26200.8655]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\alexa\Documents\GitHub\FoodStore_TrabajoIntegrador_Prog2_UTN>
onMessages -cp C:\Users\alexa\AppData\Roaming\Code\User\workspaceStorage\
2_UTN_5ae62431\bin Main "

=====
|| SISTEMA DE PEDIDOS (FOOD STORE) ||
=====
|| 1. Categorías ||
|| 2. Productos ||
|| 3. Usuarios ||
|| 4. Pedidos ||
|| 0. Salir ||
=====
Seleccione una opcion:
```

Al iniciar el programa, el usuario accede al **menú principal**, desde el cual puede dirigirse a cualquiera de los cuatro módulos del sistema o finalizar la ejecución.

Listado de productos

```

=====
Seleccione una opcion: 1
=====
|| Usted selecciono: Listar productos ||
=====
¿Deseas filtrar por categoría? (s/n)
n
ID: 1
Nombre: Samsung S24 Ultra
Precio: $1200,00
Stock: 15
Categoría: Tecnologia
-----
ID: 2
Nombre: Sofa-cama
Precio: $200,00
Stock: 30
Categoría: Hogar
-----
Presione Enter para continuar...

```

Una vez cargadas las categorías y los productos del catálogo, el módulo correspondiente permite visualizar todas las entidades activas, mostrando para cada una su **identificador**, **nombre**, **precio**, **stock** y **categoría** asociada.

Creación de un usuario

```

Seleccione una opcion: 3
=====
||          GESTION DE USUARIOS          ||
=====
|| 1. Listar usuarios                    ||
|| 2. Crear usuario                      ||
|| 3. Editar usuario                    ||
|| 4. Eliminar usuario                  ||
|| 0. Volver al menu principal          ||
=====
Seleccione una opcion: 2
=====
|| Usted selecciono: Crear usuario ||
=====
Ingrese el nombre del usuario:

```

El flujo de alta de un usuario solicita secuencialmente los datos requeridos, **nombre**, **apellido**, **correo electrónico**, **celular**, **contraseña** y **rol**, validando que ninguno de ellos sea nulo o se encuentre vacío y que el correo electrónico no esté duplicado en el sistema.

Creación de un pedido

```

=====
Seleccione una opcion: 4
=====
||      GESTION DE PEDIDOS      ||
=====
|| 1. Listar pedidos           ||
|| 2. Crear pedido con detalles ||
|| 3. Actualizar estado / forma de pago ||
|| 4. Eliminar pedido         ||
|| 0. Volver al menu principal ||
=====
Seleccione una opcion: 2
=====
|| Usted selecciono: Crear pedido ||
=====
Lista de usuarios:
ID: 1
Nombre: Alexander
Apellido: Sequera
Mail: alex@gmail.com
Rol: USUARIO
-----
0. Salir.
Ingrese el id del usuario o (0) para salir:

```

El módulo de **Pedidos** permite construir una orden seleccionando el usuario solicitante, agregando uno o más productos con sus respectivas cantidades y eligiendo la **FormaPago**. Al confirmar la operación, el sistema descuenta automáticamente el **stock** de los productos involucrados, calcula el **total** y registra el pedido con estado **PENDIENTE**.

Manejo de error: stock insuficiente

```

-----
Productos disponibles con stock:
1 - Samsung S24 Ultra | precio: $1200.0 | disponible: 15
2 - Sofa-cama | precio: $200.0 | disponible: 30
0. Finalizar selección de productos.
Ingrese el id del producto para agregar al pedido o (0) para finalizar: 1
-----
Producto seleccionado: Samsung S24 Ultra
Ingrese la cantidad a agregar al pedido: 16
-----
No hay suficiente stock disponible para esa cantidad.
Stock total del producto: 15
Podés agregar hasta 15 más.

```

Al intentar crear un pedido cuya cantidad solicitada supera el stock disponible, el sistema lanza una **StockInsuficienteException**, capturada por la **capa de interfaz**, que informa el problema al usuario sin interrumpir la ejecución del programa.

Manejo de error: dato duplicado

```

Seleccione una opcion: 1
=====
||    GESTION DE CATEGORIAS    ||
=====
|| 1. Listar categorías      ||
|| 2. Crear categoría       ||
|| 3. Editar categoría      ||
|| 4. Eliminar categoría    ||
|| 0. Volver al menu principal ||
=====
Seleccione una opcion: 2
=====
|| Usted selecciono: Crear categorías ||
=====
Ingrese el nombre de la nueva categoría: Hogar
-----
Ingrese una descripción de la nueva categoría: Producto de casa
-----
¿Está seguro que desea crear esta categoría? (s/n): s
Ya existe una categoría con ese nombre.

```

De manera análoga, el intento de registrar una categoría o un usuario con un nombre o correo electrónico ya existente produce una **DataDuplicadaException**, traducida en un mensaje claro que invita al usuario a corregir su entrada.

Dificultades y Soluciones

A lo largo del desarrollo del sistema surgieron dificultades de naturaleza **técnica**, **organizacional** y **conceptual**. A continuación se describen las más significativas y las estrategias aplicadas para resolverlas.

1. Traducción del diseño UML al código fuente

Si bien el sistema partió de un diseño previo en **UML**, el diagrama no contemplaba todos los aspectos prácticos de la implementación, como la organización en paquetes, las firmas de los métodos o el manejo concreto de las relaciones entre clases. Para resolver estas dudas se recurrió a la **documentación oficial de Java**, bibliografía técnica y herramientas de **inteligencia artificial**, utilizadas como apoyo durante la codificación. Las soluciones obtenidas fueron analizadas y adaptadas al contexto del proyecto antes de incorporarse, lo que permitió enriquecer el diseño original con paquetes no contemplados inicialmente, como **exception** y **utils**.

2. Detección y corrección de errores

Durante la ejecución aparecieron errores de distinta naturaleza, desde fallos críticos hasta comportamientos sutiles que contradecían las reglas de negocio sin generar excepciones visibles. La estrategia adoptada combinó **pruebas manuales sistemáticas** con el apoyo de herramientas de **inteligencia artificial** para revisar fragmentos de código sospechosos. Esta combinación permitió detectar inconsistencias en la lógica de validación, omisiones en el filtrado de registros eliminados y casos límite que no habían sido contemplados inicialmente.

3. Adopción de una arquitectura profesional

Se decidió no limitarse a una organización mínima del código, sino aplicar principios propios de entornos profesionales, como la **separación en capas**, la **inyección de dependencias**, la **centralización de mensajes**, el **borrado lógico** y el uso de **excepciones personalizadas**. La incorporación de estos patrones implicó una curva de aprendizaje, ya que muchos no son habituales en proyectos académicos iniciales. Para abordarla se consultó bibliografía y ejemplos de proyectos reales, lo que permitió comprender el propósito de cada patrón antes de implementarlo.

4. Modelado de la lógica de negocio

Definir las **reglas de negocio** fue uno de los aspectos más demandantes del proyecto. Surgieron dudas concretas, como cuándo descontar el stock de un pedido, qué transiciones de estado debían permitirse o cómo preservar la información histórica al eliminar un producto o un usuario. Cada una de estas cuestiones requirió discusión en equipo y revisión de la consigna para arribar a decisiones consistentes. La solución adoptada para varios de estos problemas, en particular el **borrado lógico**, permitió resolver simultáneamente múltiples cuestiones relacionadas con la **integridad referencial**.

5. Repaso de conceptos de Java y POO

Algunos conceptos teóricos de la **Programación Orientada a Objetos** resultaban claros desde el punto de vista conceptual, pero su aplicación práctica requirió un repaso sostenido del material de la asignatura y de la documentación del lenguaje. Casos como la elección entre una **clase abstracta** y una **interfaz**, o la correcta sobreescritura de los métodos **equals()** y **hashCode()**, exigieron revisar nuevamente estos conceptos para implementarlos de manera adecuada.

6. Coordinación del trabajo en equipo

Mantener la coherencia del código durante el desarrollo colaborativo fue uno de los desafíos principales del proyecto. Cada integrante avanzó sobre módulos diferentes, lo que generó criterios divergentes en aspectos como la nomenclatura, el formato de los mensajes o la estructura de los métodos. La solución consistió en establecer **convenciones comunes** desde el inicio, mantener una **comunicación constante** y realizar **revisiones cruzadas** del código antes de integrar los avances. Este proceso permitió unificar el estilo del sistema y fortalecer la comprensión de los conceptos mediante el intercambio entre los integrantes del equipo.

7. Interpretación de la consigna

La consigna del trabajo presentaba múltiples requerimientos distribuidos a lo largo del documento, cuya jerarquía y orden de implementación no resultaban evidentes en una primera lectura. Para abordar esta dificultad, además de la lectura tradicional del enunciado, se recurrió al **uso de herramientas de inteligencia artificial como apoyo a la interpretación**: se extrajeron los puntos centrales, se reorganizaron en bloques temáticos y se estableció un orden de desarrollo coherente. Este proceso permitió identificar dependencias entre requerimientos —por ejemplo, la necesidad de definir las entidades antes de avanzar con los servicios— y planificar el trabajo de manera más eficiente. La

interpretación obtenida fue validada por todos los integrantes del equipo antes de comenzar la implementación, garantizando que el desarrollo se ajustara fielmente a lo solicitado.

Anexos

Bibliografía / Webgrafía

- [Microsoft. ¿Qué es Java?](#) — para la definición general, historia, usos y características del lenguaje.
- [Amazon Web Services \(AWS\). ¿Qué es Java?](#) — para el funcionamiento de la JVM, el modelo compilador/intérprete y los aspectos de seguridad.
- [GeeksforGeeks. Conceptos de programación orientada a objetos \(POO\) en Java.](#)
- [OpenWebinar. Introducción a POO en Java: Objetos y clases](#)
- [Oracle. The Collection Interface \(The Java™ Tutorials\).](#)
- [Oracle. Enum Types \(The Java™ Tutorials\).](#)
- [CodeGym. Excepciones en Java](#) — para definición, jerarquía, palabras clave y patrones de manejo de excepciones.

Links de video

<https://www.youtube.com/watch?v=9dOQyNm13NU>